Advanced Programming for Scientists

Lecture 4

Code Timing and Bottleneck Identification

v1.0

Prof. Mateusz Bawaj

A.D. 1308 unipg
UNIVERSITÀ DEGLI STUDI DI PERUGIA

Motivation

Scientific software often evolves from quick exploratory scripts into tools used for large simulations or data processing.

Typical situation in research:

- a prototype written in Python
- datasets or parameter space increase
- execution time becomes prohibitive

Important principle:

Optimization should start only after measuring performance.

Premature optimization:

- wastes time
- complicates code
- often targets the wrong part of the program.

Where time is spent in scientific codes

Typical computational patterns in scientific workflows:

- numerical simulations (ODE, PDE solvers)
- large data analysis pipelines
- Monte-Carlo simulations
- machine learning training loops

Common bottlenecks:

- nested loops
- expensive numerical routines
- memory access patterns
- disk I/O
- communication (parallel computing)

Pareto principle

“...for many outcomes, roughly 80% of consequences come from 20% of causes.”

Observation from many real systems:

Often 90% of runtime is spent in a very small fraction of the code.

This can be linked to the Pareto principle or 80/20 rule in performance analysis.

Example from scientific computing

What is your experience with software time execution?

Basic Timing Tools (python)

The simplest way to measure runtime is wall clock timing.

Python example:

```python
import time

start = time.time()

run_simulation()

end = time.time()
print("Execution time:", end - start)
```

Advantages:

- simple
- useful for quick tests

Limitations:

- coarse measurement
- not suitable for small functions.

High-resolution timing (python)

Better timing uses high precision timers.

Example:

```
import time

start = time.perf_counter()
run_simulation()
end = time.perf_counter()

print(end - start)
```

`perf_counter()`:

- high-resolution clock
- includes sleep time
- recommended for benchmarking.

Timing inside Jupyter

In research environments, Jupyter notebooks are widely used.

Magic commands:

`%time` – Time the execution of a single statement

`%timeit` – Time repeated execution of a single statement for more accuracy

Example:

```
%timeit compute_model()
```

`timeit`:

- runs code multiple times
- computes statistical averages
- reduces noise from system variability.

Useful for comparing algorithm variants.

Profiling vs Timing

Timing answers:

How long does my program take?

Profiling answers:

Where exactly does it spend time?

Profiling tools measure:

- function call time
- number of calls
- cumulative execution time.

This allows identifying **bottlenecks**.

Python profiler (cProfile)

Example usage:

```
$ python -m cProfile -s tottime myscript.py
```

Typical output:

```
         95776 function calls (93121 primitive calls) in 0.131 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      586    0.017    0.000    0.017    0.000 _inspect.py:65(getargs)
      526    0.017    0.000    0.017    0.000 <frozen importlib._bootstrap>:179(_get_module_lock)
  191/131    0.016    0.000    0.025    0.000 typing.py:306(inner)
```

Interpretation:

- tottime: time inside function
- cumtime: time including subcalls

Focus optimization on functions with **largest cumulative time**.

Python profiler (cProfile)

Generate profile output file:

```
$ python -m cProfile -o myscript.prof myscript.py
```

Visualization tools

Profiling results become clearer when visualized.

Common tools:

- `snakeviz`
- `gprof2dot`
- flame graphs

Example:

```
$ snakeviz myscript.prof
```

Visualization helps identify:

- deep call stacks
- heavy functions
- unexpected performance issues.

Basic Timing in C/C++

In compiled languages such as C and C++, timing is typically performed using system clocks provided by the standard library.

Simple example using the C standard library:

```c
#include <stdio.h>
#include <time.h>

int main() {
    clock_t start = clock();

    run_simulation();

    clock_t end = clock();
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Execution time: %f seconds\n", time_spent);
}
```

Basic Timing in C/C++

Key ideas:

- `clock()` returns CPU time used by the program
- difference between `end` and `start` gives elapsed CPU cycles
- divide by `CLOCKS_PER_SEC` to obtain seconds

Limitations:

- measures **CPU time**, not wall-clock time
- resolution may be limited on some systems

Useful for quick measurements in scientific kernels.

High-Resolution Timing with `<chrono>` (C++)

C++ provides high-precision timers through the <chrono> library.

Example:

```
#include <iostream>
#include <chrono>

int main() {

    auto start = std::chrono::high_resolution_clock::now();

    run_simulation();

    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> elapsed = end - start;

    std::cout << "Execution time: "
              << elapsed.count()
              << " seconds\n";
}
```

High-Resolution Timing with `<chrono>` (C++)

Advantages:

high precision timing

portable across systems

measures wall-clock time

Common clocks:

- `system_clock`
- `steady_clock`
- `high_resolution_clock`

Recommended choice for benchmarking code seriously.

Memory Profiling

Performance problems are not always CPU related.

Large datasets may create issues such as:

- memory exhaustion
- swapping to disk
- cache inefficiency

Tools:

- `memory_profiler`
- `tracemalloc`

Understanding memory usage is essential for **large** scientific datasets.

Memory Profiling

```
memory_profiler:
```

Example:

```
from memory_profiler import profile

@profile
def my_function():
    print("Hello")
```

Algorithmic Bottlenecks

The largest improvements usually come from **algorithmic changes**, not code tweaks.

Example:

Algorithm	**Complexity**
Naive search	$O(N^2)$
Efficient algorithm	$O(N \log N)$

Changing algorithm often yields **orders of magnitude improvement**.

Example in physics:

- FFT vs direct convolution
- matrix factorization vs brute force solver.

Vectorization

Vectorization is key in scientific Python.

Instead of:

```
for i in range(N):
    y[i] = a*x[i] + b
```

Use:

```
y = a*x + b
```

Advantages:

- uses optimized C routines
- reduces interpreter overhead
- improves memory locality.

When Python becomes a bottleneck

In some cases Python overhead becomes dominant.

Typical cases:

- inner loops
- real-time simulations
- repeated numerical kernels

Solutions:

- NumPy / SciPy
- Numba JIT compilation
- Cython
- C++ extensions.

This connects to the programming language spectrum discussed in Lesson 1.

Profiling workflow

Recommended workflow for scientific projects:

1) Write correct code first
2) Measure runtime
3) Profile execution
4) Identify bottleneck
5) Optimize only critical parts
6) Re-measure performance.

This cycle is called **performance engineering**.

Preparing for Parallelization

Profiling is also necessary before using:

- GPUs
- HPC clusters
- distributed computing

Key rule:

> **Parallelizing inefficient code produces inefficient parallel code.**

Always optimize the **serial algorithm first**.

Connection to Next Lessons

Next steps in performance improvement:

1) Compiled acceleration
   - Numba
   - C/C++
2) Parallel computing
   - multiprocessing
   - MPI
3) Hardware acceleration
   - GPU computing
   - FPGA pipelines

Suggested Practical Exercise

Measure runtime of three implementations:

1)Python loop

2)NumPy vectorized code

3)Numba-compiled function

Task:

- measure runtime with timeit
- profile with cProfile
- identify the bottleneck.

Submit results through GitHub issues in the course repository.